



Road Reorganization (roads)

Das römische Heer hat kürzlich neue Kutschen erhalten, die deutlich mehr Menschen transportieren können. Leider bedeutet dies auch, dass die Kutschen zu gross sind und oft stecken bleiben, wenn Gegenverkehr herrscht. Um dies zu verhindern, beschliesst der Kaiser, alle Strassen im Reich zu Einbahnstrassen zu erklären.

Der Kaiser besucht jedoch gerne und oft seine Heimatstadt und möchte sicherstellen, dass er von dort aus schnell nach Rom zurückkehren kann, falls etwas Wichtiges aufkommt. Um dies zu gewährleisten, möchte er dafür sorgen, dass der kürzeste Weg von seiner Heimatstadt nach Rom nicht länger wird als bisher. Ausserdem möchte er sicherstellen, dass er in seine Heimatstadt zurückkehren kann, aber dies muss nicht so schnell wie möglich geschehen.

Das Römische Reich umfasst N Städte und M bidirektionale Strassen. Die Heimatstadt des Kaisers ist die Stadt 0 und Rom die Stadt $N - 1$. Formal möchte er für jede Strasse eine Richtung festlegen. Er möchte sicherstellen, dass der kürzeste Weg von der Stadt 0 zur Stadt $N - 1$ nicht länger ist als vor der Umstellung auf Einbahnstrassen. Ausserdem möchte er sicherstellen, dass es weiterhin einen Weg von der Stadt $N - 1$ zurück zur Stadt 0 gibt. Deine Aufgabe ist es, die Strassen entsprechend auszurichten oder den Kaiser wissen zu lassen, dass dies unmöglich ist.

$\implies$ Es ist garantiert, dass jede Stadt im Römischen Reich von jeder anderen Stadt aus erreichbar ist. Beachte, dass dies nach der Ausrichtung der Strassen nicht mehr der Fall sein muss.

Beachte, dass du auch dann noch Punkte erhältst, wenn du die Strassen so ausrichtest, dass du von 0 nach $N - 1$ und zurück fahren kannst, ohne dabei die Länge einer der Fahrten unbedingt zu minimieren.

Implementierung

Du musst eine einzelne `.cpp` Quelldatei einreichen.



Unter den Anhängen zu dieser Aufgabe findest du eine Vorlage `roads.cpp` mit einer Beispielimplementierung.

Du musst die folgende Funktion implementieren:

C++

```
vector<pair<int, int>> orient_roads(int N, int M, vector<int> A,
vector<int> B, vector<int> W)
```

- Die Ganzzahl N stellt die Anzahl der Städte dar.
- Die Ganzzahl M stellt die Anzahl der Strassen dar.
- Die Arrays A , B und W beschreiben die Strassen des römischen Reiches: Es gibt eine Strasse zwischen Stadt $A[i]$ und Stadt $B[i]$ mit der Länge $W[i]$.
- Wenn es eine Möglichkeit gibt, die Strassen auf diese Weise auszurichten, sollte die Funktion einen Vektor von Paaren der Länge M zurückgeben. Hier bedeutet das i -te Paar $\{F, T\}$, dass es eine Strasse gibt, die von F nach T verläuft.

- Wenn es keine Möglichkeit gibt, die Strassen auszurichten, sollte die Funktion einen leeren Vektor zurückgeben.

Es ist garantiert, dass höchstens eine Strasse zwischen zwei Städten existiert und dass für alle i $A[i] \neq B[i]$ gilt. Gibt die Funktion keinen Vektor der Länge 0 oder M zurück, gilt dies als falsche Antwort. Alle in der Eingabe genannten Strassen müssen im Ausgabevektor genau einmal vorkommen, und es dürfen keine weiteren Strassen vorhanden sein. Die Reihenfolge der Strassen im Ausgabevektor spielt keine Rolle.

Beispielgrader

Das Verzeichnis der Aufgabe enthält eine vereinfachte Version des Jury-Graders, die Du verwenden kannst, um Deine Lösung lokal zu testen. Der vereinfachte Grader liest die Eingangsdaten aus der Datei `stdin`, ruft die Funktion `orient_roads` auf und schreibt schliesslich die Ausgabe in die Datei `stdout` mit folgendem Format.

Die Eingabe besteht aus $M + 1$ Zeilen, die Folgendes enthalten:

- Zeile 1: die Ganzzahlen N und M .
- Zeile $2 + i$ ($0 \leq i < M$): die Ganzzahlen A_i , B_i und W_i .

Die Ausgabe besteht aus entweder 1 oder $M + 1$ Zeilen, die Folgendes enthalten:

- Zeile 1: **Yes** oder **No**, um anzugeben, ob es möglich ist, die Strassen auf solche Weise auszurichten.
- Wenn die Antwort **Yes** ist: Zeile $2 + i$ ($0 \leq i < M$): F_i und T_i , um anzugeben, dass die Strasse zwischen diesen beiden Städten von F_i nach T_i verlaufen sollte.

In der Ausgabe werden die Strassenausrichtungen in der Reihenfolge gedruckt, in der sie von der Funktion zurückgegeben werden.

Einschränkungen

- $3 \leq N \leq 100\,000$.
- $N - 1 \leq M \leq 100\,000$.
- $0 \leq A_i, B_i \leq N - 1$ and $A_i \neq B_i$.
- $1 \leq W_i \leq 1\,000\,000\,000$.
- Es ist garantiert, dass jede Stadt im Römischen Reich von jeder anderen Stadt aus erreichbar ist.

Punktevergabe

Dein Programm wird anhand einer Reihe von Testfällen getestet, die nach Teilaufgaben gruppiert sind. Um die volle Punktzahl zu erhalten, die einer Teilaufgabe zugeordnet ist, musst du alle darin enthaltenen Testfälle korrekt lösen. Ist die Lösung gültig, aber nicht optimal, erhältst du für diese Teilaufgabe die Hälfte der Punkte.

- **Teilaufgabe 0 [0 Punkte]**: Beispiel-Testfälle.
- **Teilaufgabe 1 [6 Punkte]**: $\max(N, M) \leq 20$.
- **Teilaufgabe 2 [11 Punkte]**: $M \leq N$.
- **Teilaufgabe 3 [17 Punkte]**: Es ist garantiert, dass es eine direkte Strasse zwischen Stadt 0 und Stadt $N - 1$ gibt.
- **Teilaufgabe 4 [18 Punkte]**: $\max(N, M) \leq 2000$.
- **Teilaufgabe 5 [17 Punkte]**: $W_i = 1$.
- **Teilaufgabe 6 [31 Punkte]**: Keine zusätzlichen Einschränkungen.



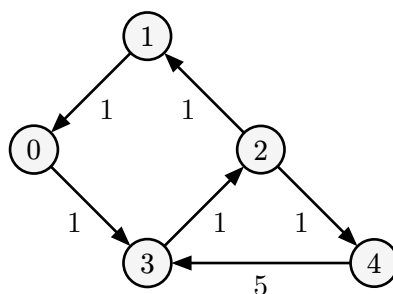
Du kannst die Hälfte der Punkte für einen Testfall erhalten, wenn der kürzeste Weg von Stadt 0 nach $N - 1$ nach der Neuordnung der Strassen nicht dieselbe Länge hat wie der

kürzeste Weg von Stadt 0 nach $N - 1$ vor der Neuordnung der Strassen, selbst wenn es keine Möglichkeit gab, die Strassen so auszurichten, dass der Kaiser völlig zufrieden war.

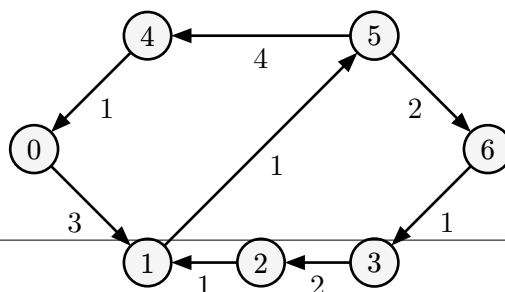
Beispiele

stdin	stdout
5 6 0 1 1 1 2 1 0 3 1 3 2 1 2 4 1 3 4 5	Yes 1 0 2 1 0 3 3 2 2 4 4 3
7 8 0 1 3 1 2 1 2 3 2 3 6 1 0 4 1 4 5 4 5 6 2 1 5 1	Yes 0 1 2 1 3 2 6 3 4 0 5 4 5 6 1 5
8 9 0 1 3 1 2 4 2 4 2 0 3 6 3 4 1 4 5 3 5 6 1 6 7 2 5 7 3	No

Erklärung



Im ersten Beispiel gelangen wir von Stadt 0 nach 4 über $0 \rightarrow 3 \rightarrow 2 \rightarrow 4$. Dies ist ein Pfad mit der Länge 3, der Mindestlänge vor der Umleitung der Strassen. Wir können auch über $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$ zurückkommen. Die Länge dieses Pfades spielt keine Rolle.



Im zweiten Beispiel gelangen wir mit der Route $0 \rightarrow 1 \rightarrow 5 \rightarrow 6$ von 0 nach 6 und mit $6 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 5 \rightarrow 4 \rightarrow 0$ zurück.

Im dritten Beispiel lässt sich zeigen, dass es keine Strassenführung gibt, die den Kaiser zufriedenstellen würde.